

SUMMER CAMP — ASSIGNMENT 5

C Programming — Arrays & String Error Detection

Submitted By

Name	yogesh swami
Class	B.Tech AI & Data Science — Section c
Roll No.	25EARAD189
Total Problems	5 Programs
Assignment	Assignment 5 — Arrays & String Error Detection

ASSIGNMENT OVERVIEW

Problem	Focus Area	Error Types	Count
P1	Array out-of-bounds, invalid index types, integer division, format specifier, array assignment	Syntax, Semantic, Runtime, Logical	16
P2	Off-by-one, logical errors, buffer overflow, invalid index types, missing semicolon, array assignment	Syntax, Semantic, Runtime, Logical	20
P3	Logical errors in finding lowest, off-by-one, buffer overflow, scanf with array, array assignment	Syntax, Semantic, Runtime, Logical	19
P4	Logical errors in max/average, buffer overflow, invalid indices, missing semicolons, null terminator	Syntax, Semantic, Runtime, Logical	21
P5	Comprehensive array errors — logical, runtime, semantic, and syntax across all common patterns	All types	22

Problem 1 — Array Basics — Marks & Student Data

Description:

Covers fundamental array errors using a student marks system: off-by-one in loops, writing beyond array bounds, float/string as array index, integer division for float result, wrong format specifier, missing & in scanf, direct array assignment, and char array index used with string literal.

Code (with intentional errors):

```
int marks[5]={80,90,70,60,50} // missing semicolon

for(int i=0;i<=5;i++)          // i<=5 reads marks[5] - out-of-bounds

average = total / 5;           // integer division loses decimal

printf("Average = %d", average); // float printed with %d

scores[5] = 60;                // out-of-bounds on scores[5]

strcpy(subject, "Programming"); // 11 chars in subject[5]

attendance["one"] = 1;          // string index

attendance[2.5] = 1;            // float index

for(int i=0;i<=5;i++) attendance // off-by-one

scanf("%d", arr[i]);            // missing & - should be &arr[i]

grade[0] = "A";                 // string literal to char element

if(prices[2] = 300)              // assignment instead of ==

inventory = prices;              // array to array direct assignment

printf("%d", salary[0]);         // float printed with %d

city = "Jaipur";                // string assigned to char array

for(int i=0;i<=5;i++) result    // off-by-one
```

Error Analysis — 16 Errors Found:

Line No.	Error Type	Error Description	Correction
6	Syntax	Missing semicolon after array initializer 'int marks[5] = {...}'	<i>int marks[5] = {80,90,70,60,50};</i>
12	Runtime	Loop i<=5 reads marks[5] on a size-5 array — out-of-bounds access	<i>for(int i=0; i<5; i++)</i>
20	Logical	average = total/5 performs integer division; decimal part lost for float variable	<i>average = (float)total / 5;</i>
22	Semantic	printf("%d", average) — float variable printed with integer format %d	<i>printf("Average = %.2f\n", average);</i>
31	Runtime	scores[5] = 60 writes to index 5 on scores[5] — out-of-bounds (valid: 0-4)	<i>Declare scores[6] or don't assign beyond index 4</i>
41	Runtime	strcpy of 11 chars into subject[5] — buffer overflow	<i>char subject[15]; strcpy(subject, "Programming");</i>
45	Semantic	attendance["one"] — string used as array index; must be integer	<i>attendance[0] = 1;</i>
47	Semantic	attendance[2.5] — float used as array index; must be integer	<i>attendance[2] = 1;</i>
54	Runtime	for(i<=5) loop on attendance[5] accesses attendance[5] — out-of-bounds	<i>for(int i=0; i<5; i++)</i>
		scanf("%d", arr[i]) — missing &	

60	Semantic	operator; should pass address of arr[i]	<i>scanf("%d", &arr[i]);</i>
72	Semantic	grade[0] = "A" — assigns string literal (char*) to char element; needs single quotes	<i>grade[0] = 'A';</i>
77	Logical	if(prices[2] = 300) — assignment instead of comparison; always true	<i>if(prices[2] == 300)</i>
82	Semantic	inventory = prices — cannot assign array to array directly in C	<i>Use memcpy(inventory, prices, sizeof(prices)); or loop copy</i>
88	Semantic	printf("%d", salary[0]) — float element printed with %d format	<i>printf("%.2f\n", salary[0]);</i>
93	Semantic	city = "Jaipur" — cannot assign string to char array with =; use strcpy()	<i>strcpy(city, "Jaipur");</i>
98	Runtime	for(i<=5) on result[5] writes result[5] — out-of-bounds	<i>for(int i=0; i<5; i++)</i>

Problem 2 — Array Operations — Numbers & Fees

Description:

Array operations program with errors in loop bounds, sum accumulation, finding highest mark (wrong condition), out-of-bounds attendance write, buffer overflow in strings, invalid index types, missing & in scanf, array-to-array assignment, and wrong format specifier for float.

Code (with intentional errors):

```
for(int i=0;i<=10;i++) numbers // off-by-one

sum = numbers[i];                // replaces instead of accumulates

average = sum / 10;              // integer division

printf("Average = %d", average); // float with %d

if(marks[i] < highest)           // logical: finds LOWEST not highest

attendance[5]=1 on attendance[5] // out-of-bounds

for(int i=0;i<=5;i++) present   // off-by-one

strcpy(subject, "ComputerScience"); // 15 chars in subject[10]

branch = "IT";                  // array assignment with =

college not null-terminated     // no '\0' after college[4]

fees["two"] = 50000;             // string index

fees[3.5] = 60000;              // float index

scanf("%d", result[i]);         // missing &

if(stock[2] = 300)              // assignment not ==

backup = stock;                 // array-to-array assignment

printf("%d", salary[0]);        // float with %d

for(i<=5) inventory            // off-by-one

strcpy(city,"JaipurCity");      // 10 chars in city[8]

count = inventory;             // array assignment

printf("End...")               // missing semicolon
```

Error Analysis — 20 Errors Found:

Line No.	Error Type	Error Description	Correction
16	Runtime	Loop <code>i<=10</code> reads <code>numbers[10]</code> on size-10 array — off-by-one out-of-bounds	<code>for(int i=0; i<10; i++)</code>
22	Logical	<code>sum = numbers[i]</code> replaces <code>sum</code> each iteration instead of accumulating	<code>sum += numbers[i];</code>
27	Logical	<code>average = sum/10</code> performs integer division; decimal result lost	<code>average = (float)sum / 10;</code>
29	Semantic	<code>printf("%d", average)</code> — float printed with integer format <code>%d</code>	<code>printf("Average = %.2f\n", average);</code>
39	Logical	<code>if(marks[i] < highest)</code> updates <code>highest</code> when smaller found — finds LOWEST not highest	<code>if(marks[i] > highest) highest = marks[i];</code>
52	Runtime	<code>attendance[5]=1</code> writes to index 5 on <code>attendance[5]</code> — out-of-bounds	Declare <code>attendance[6]</code> or don't write to index 5
57	Runtime	Loop <code>i<=5</code> on <code>attendance[5]</code> reads <code>attendance[5]</code> — out-of-bounds	<code>for(int i=0; i<5; i++)</code>
63	Runtime	<code>strcpy</code> of 15-char string into <code>subject[10]</code> — buffer overflow	<code>char subject[20]; strcpy(subject, "ComputerScience");</code>
69	Semantic	<code>branch = "IT"</code> — cannot assign string literal to char array with <code>=</code>	<code>strcpy(branch, "IT");</code>
76	Runtime	<code>college[0..4]</code> set but no null terminator — <code>printf("%s")</code> prints	Add: <code>college[5] = '\0';</code> or declare <code>college[6]</code>

		garbage beyond	
80	Semantic	fees["two"] — string used as array index; must be integer	<i>fees[1] = 50000;</i>
82	Semantic	fees[3.5] — float used as array index; must be integer	<i>fees[3] = 60000;</i>
91	Semantic	scanf("%d", result[i]) — missing & operator for scanf address argument	<i>scanf("%d", &result[i]);</i>
100	Logical	if(stock[2] = 300) — assignment always true, not comparison	<i>if(stock[2] == 300)</i>
105	Semantic	backup = stock — cannot copy array with = in C	<i>memcpy(backup, stock, sizeof(stock));</i>
110	Semantic	printf("%d", salary[0]) — float element printed with %d format	<i>printf("%.2f\n", salary[0]);</i>
115	Runtime	Loop i<=5 on inventory[5] writes inventory[5] — out-of-bounds	<i>for(int i=0; i<5; i++)</i>
121	Runtime	strcpy(city, "JaipurCity") — 10 chars + null into city[8] — buffer overflow	<i>char city[15]; strcpy(city, "JaipurCity");</i>
126	Semantic	count = inventory — cannot assign array to array with =	<i>memcpy(count, inventory, sizeof(inventory));</i>
131	Syntax	Missing semicolon after final printf statement	<i>printf("Program Ended\n");</i>

Problem 3 — Student Result Analysis

Description:

Student result analysis program. Key errors: integer division for average, wrong condition for finding lowest value, off-by-one attendance loop, buffer overflows, scanf with wrong format for arrays, direct array assignment, invalid index types, and missing return semicolon.

Code (with intentional errors):

```
average = total / 5;           // integer division

if(marks[i] > lowest)          // should be < to find lowest

for(int i=0;i<=5;i++) present // off-by-one on attendance[5]

strcpy(subject[8],"Programming") // 11 chars in subject[8]

scanf("%s", &branch)           // & not needed for char array

fees[5] = 100000;               // out-of-bounds on fees[5]

if(stock[2] = 30)               // assignment not ==

backup = stock;                 // array assignment

printf("%d", salary[0]);        // float with %d

scanf("%d", result[i]);         // missing &

city = "Jaipur";                // array assignment with =

inventory["one"]=100;           // string index

inventory[2.5]=200;             // float index

for(i<=5) inventory             // off-by-one

college[0..4] no null term      // missing '\0'

count = inventory;              // array assignment

strcpy(course[6],"Computer");   // 8 chars in course[6]

for(i<=5) numbers               // off-by-one

return 0                         // missing semicolon
```


Line No.	Error Type	Error Description	Correction
21	Logical	average = total/5 — integer division truncates decimal for float result	<i>average = (float)total / 5;</i>
35	Logical	if(marks[i] > lowest) updates lowest when LARGER found — finds highest, not lowest	<i>if(marks[i] < lowest) lowest = marks[i];</i>
51	Runtime	Loop i<=5 on attendance[5] reads attendance[5] — out-of-bounds	<i>for(int i=0; i<5; i++)</i>
58	Runtime	strcpy of 11 chars into subject[8] — buffer overflow	<i>char subject[15]; strcpy(subject, "Programming");</i>
63	Semantic	scanf("%s", &branch) — char array already decays to pointer; & is incorrect here	<i>scanf("%s", branch);</i>
72	Runtime	fees[5] = 100000 writes to index 5 on fees[5] — out-of-bounds (valid: 0-4)	<i>Declare fees[6] or don't assign index 5</i>
77	Logical	if(stock[2] = 30) — assignment instead of comparison; always true	<i>if(stock[2] == 30)</i>
82	Semantic	backup = stock — cannot assign arrays with = in C	<i>memcpy(backup, stock, sizeof(stock));</i>
87	Semantic	printf("%d", salary[0]) — float printed with integer format	<i>printf("%.2f\n", salary[0]);</i>
92	Semantic	scanf("%d", result[i]) — missing & for scanf address	<i>scanf("%d", &result[i]);</i>

99	Semantic	city = "Jaipur" — cannot assign string to char array with =	<i>strcpy(city, "Jaipur");</i>
103	Semantic	inventory["one"] — string used as array index	<i>inventory[0] = 100;</i>
105	Semantic	inventory[2.5] — float used as array index	<i>inventory[2] = 200;</i>
112	Runtime	Loop i<=5 on inventory[5] accesses inventory[5] — out-of-bounds	<i>for(int i=0; i<5; i++)</i>
115	Runtime	college[0..4] written but no null terminator added — printf prints garbage	<i>college[5] = '\0'; or declare char college[6]</i>
120	Semantic	count = inventory — cannot copy array with =	<i>memcpy(count, inventory, sizeof(inventory));</i>
124	Runtime	strcpy(course, "Computer") — 8 chars + null into course[6] — buffer overflow	<i>char course[10]; strcpy(course, "Computer");</i>
130	Runtime	Loop i<=5 on numbers[5] writes numbers[5] — out-of-bounds	<i>for(int i=0; i<5; i++)</i>
~last	Syntax	Missing semicolon after 'return 0'	<i>return 0;</i>

Problem 4 — Sales Report System

Description:

Sales report system with array-based data. Errors: wrong comparison for highest sale (finds lowest), integer division for average, off-by-one loops, buffer overflows, string/float array indices, missing & in scanf, array assignment with =, float printed with %d, null terminator missing, and wrong format specifier.

Code (with intentional errors):

```
averageSales = totalSales/5;    // integer division

printf("Average = %d", avg);    // float with %d

if(sales[i] < highestSale)      // finds LOWEST not highest

monthlySales[5] = 250;          // out-of-bounds on [5]

for(i<=5) monthlySales          // off-by-one

strcpy(product[8],"LaptopComputer") // 14 chars in [8]

category = "Electronics";       // array assignment

stock["first"] = 100;            // string index

stock[2.7] = 200;                // float index

scanf("%d", orders[i]);         // missing &

if(revenue[2] = 3000)            // assignment not ==

backupRevenue = revenue;        // array assignment

printf("%d", commission[0]);    // float with %d

city = "Jaipur";                // array assignment

branch[0..3] no null term       // missing '\0'

for(i<=5) employees             // off-by-one

strcpy(department[6],"Marketing") // 9 chars in [6]

expenses = sales;                // array assignment

ratings[-1] = 5;                // negative index

printf("%s", feedback[0]);      // int with %s
```

```
printf("Report...")           // missing semicolon
```

Error Analysis — 21 Errors Found:

Line No.	Error Type	Error Description	Correction
20	Logical	averageSales = totalSales/5 — integer division; decimal lost for float result	<i>averageSales = (float)totalSales / 5;</i>
22	Semantic	printf("%d", averageSales) — float printed with %d format	<i>printf("Average Sales = %.2f\n", averageSales);</i>
29	Logical	if(sales[i] < highestSale) updates when smaller — this finds LOWEST not highest	<i>if(sales[i] > highestSale) highestSale = sales[i];</i>
41	Runtime	monthlySales[5] = 250 writes to index 5 on monthlySales[5] — out-of-bounds	<i>Declare monthlySales[6] to hold 6 elements</i>
45	Runtime	Loop i<=5 on monthlySales[5] accesses index 5 — out-of-bounds	<i>for(int i=0; i<5; i++)</i>
50	Runtime	strcpy of 14-char string into product[8] — buffer overflow	<i>char product[20]; strcpy(product, "LaptopComputer");</i>
55	Semantic	category = "Electronics" — cannot assign string to array with =	<i>strcpy(category, "Electronics");</i>
60	Semantic	stock["first"] — string literal used as array index; must be integer	<i>stock[0] = 100;</i>
62	Semantic	stock[2.7] — float used as array index; must be integer	<i>stock[2] = 200;</i>
71	Semantic	scanf("%d", orders[i]) — missing & address operator	<i>scanf("%d", &orders[i]);</i>

80	Logical	if(revenue[2] = 3000) — assignment always evaluates to 3000 (true), not comparison	<i>if(revenue[2] == 3000)</i>
85	Semantic	backupRevenue = revenue — cannot copy array with = assignment in C	<i>memcpy(backupRevenue, revenue, sizeof(revenue));</i>
90	Semantic	printf("%d", commission[0]) — float element printed with integer format %d	<i>printf("%.2f\n", commission[0]);</i>
95	Semantic	city = "Jaipur" — cannot assign string to char array with =	<i>strcpy(city, "Jaipur");</i>
100	Runtime	branch[0..3] written but no null terminator — printf("%s") prints garbage after	<i>branch[4] = '\0'; or declare branch[5]</i>
106	Runtime	Loop i<=5 on employees[5] accesses employees[5] — out-of-bounds	<i>for(int i=0; i<5; i++)</i>
112	Runtime	strcpy of 9-char string into department[6] — buffer overflow	<i>char department[15]; strcpy(department, "Marketing");</i>
117	Semantic	expenses = sales — cannot assign one array to another with =	<i>memcpy(expenses, sales, sizeof(sales));</i>
122	Runtime	ratings[-1] = 5 — negative index accesses memory before the array	<i>Use valid indices ratings[0]..ratings[4]</i>
127	Semantic	printf("%s", feedback[0]) — integer element printed with string format %s	<i>printf("%d\n", feedback[0]);</i>

~last	Syntax	Missing semicolon after final printf("Report Generated Successfully")	<i>printf("Report Generated Successfully\n");</i>

Problem 5 — Student Performance Analysis

Description:

A comprehensive student performance analysis program combining all common array error patterns: wrong comparison for highest/lowest, off-by-one bounds, out-of-bounds writes, buffer overflow, invalid indices, array assignment, missing & in scanf, wrong format specifiers, null terminator missing, and negative index.

Code (with intentional errors):

```
average = total / 10;           // integer division

printf("Average = %d", average); // float with %d

if(marks[i] < highest)          // finds LOWEST not highest

if(marks[i] > lowest)           // finds HIGHEST not lowest

attendance[10]=0 on [10]        // out-of-bounds

for(i<=10) present              // off-by-one

strcpy(subject[10],"DataStructures") // 14 chars in [10]

department = "InformationTechnology" // array assignment

fees["two"]=50000;               // string index

fees[4.5]=70000;                // float index

scanf("%d", result[i]);         // missing &

if(inventory[2] = 30)           // assignment not ==

backup = inventory;             // array assignment

printf("%d", salary[0]);        // float with %d

city = "Jaipur";               // array assignment

branch[0..3] no null term       // missing '\0'

for(i<=5) stock                 // off-by-one

strcpy(course[8],"Programming") // 11 chars in [8]

ratings[-1]=10;                 // negative index

printf("%s", feedback[0]);      // int with %s
```

```
scores = marks;                // array assignment (marks[10] -> scores[5])

printf("%d", temp[5]);          // out-of-bounds on temp[5]
```

Error Analysis — 22 Errors Found:

Line No.	Error Type	Error Description	Correction
20	Logical	average = total/10 — integer division truncates decimal part for float	<i>average = (float)total / 10;</i>
22	Semantic	printf("%d", average) — float printed with integer format %d	<i>printf("Average = %.2f\n", average);</i>
29	Logical	if(marks[i] < highest) updates when smaller — finds LOWEST instead of highest	<i>if(marks[i] > highest) highest = marks[i];</i>
39	Logical	if(marks[i] > lowest) updates when larger — finds HIGHEST instead of lowest	<i>if(marks[i] < lowest) lowest = marks[i];</i>
51	Runtime	attendance[10]=0 writes to index 10 on attendance[10] — out-of-bounds (valid 0-9)	<i>attendance[9] = 0; or declare attendance[11]</i>
55	Runtime	Loop i<=10 on attendance[10] reads index 10 — off-by-one out-of-bounds	<i>for(int i=0; i<10; i++)</i>
62	Runtime	strcpy of 14-char string into subject[10] — buffer overflow	<i>char subject[20]; strcpy(subject, "DataStructures");</i>
68	Semantic	department = "InformationTechnology" — cannot assign string to char array with =	<i>strcpy(department, "InformationTechnology"); // also increase array size to [25]</i>
73	Semantic	fees["two"] — string used as array index; must be integer	<i>fees[1] = 50000;</i>
75	Semantic	fees[4.5] — float used as array	<i>fees[4] = 70000;</i>

		index; must be integer	
84	Semantic	scanf("%d", result[i]) — missing & address operator for scanf	<i>scanf("%d", &result[i]);</i>
96	Logical	if(inventory[2] = 30) — assignment (always true) instead of comparison	<i>if(inventory[2] == 30)</i>
101	Semantic	backup = inventory — cannot copy array to array with = in C	<i>memcpy(backup, inventory, sizeof(inventory));</i>
106	Semantic	printf("%d", salary[0]) — float printed with integer format %d	<i>printf("%.2f\n", salary[0]);</i>
111	Semantic	city = "Jaipur" — cannot assign string to char array with = operator	<i>strcpy(city, "Jaipur");</i>
116	Runtime	branch[0..3] assigned but no null terminator — printf("%s") prints garbage	<i>branch[4] = '\0'; or declare branch[5]</i>
120	Runtime	Loop i<=5 on stock[5] writes stock[5] — off-by-one out-of-bounds	<i>for(int i=0; i<5; i++)</i>
126	Runtime	strcpy of 11 chars into course[8] — buffer overflow	<i>char course[15]; strcpy(course, "Programming");</i>
132	Runtime	ratings[-1] = 10 — negative index accesses memory before array start	<i>Use ratings[0]..ratings[4] only</i>
136	Semantic	printf("%s", feedback[0]) — integer printed with string format %s	<i>printf("%d\n", feedback[0]);</i>
		scores = marks — cannot assign	

140	Semantic	arrays with = (also sizes differ: marks[10] vs scores[5])	<i>memcpy(scores, marks, sizeof(scores)); // copies only first 5</i>
146	Runtime	printf("%d", temp[5]) — accesses index 5 on temp[5] — out-of-bounds	<i>temp[4] is the last valid index</i>

— *End of Assignment 5* —

yogesh swami | 25EARAD189 | AI & Data Science - c | Summer Camp